Trading Dashboard — Deployment Blueprint

How my Delta paper-trading platform is built and deployed, written so you can deploy your dashboard the same way. Live example: https://trader-abhi.duckdns.org (a single free Oracle VM, ~₹0/month).

Placeholders used throughout: `<YOUR_VM_IP>`, `<name>.duckdns.org`, `<repo>`. Never put real keys, passwords or IPs into git.

---

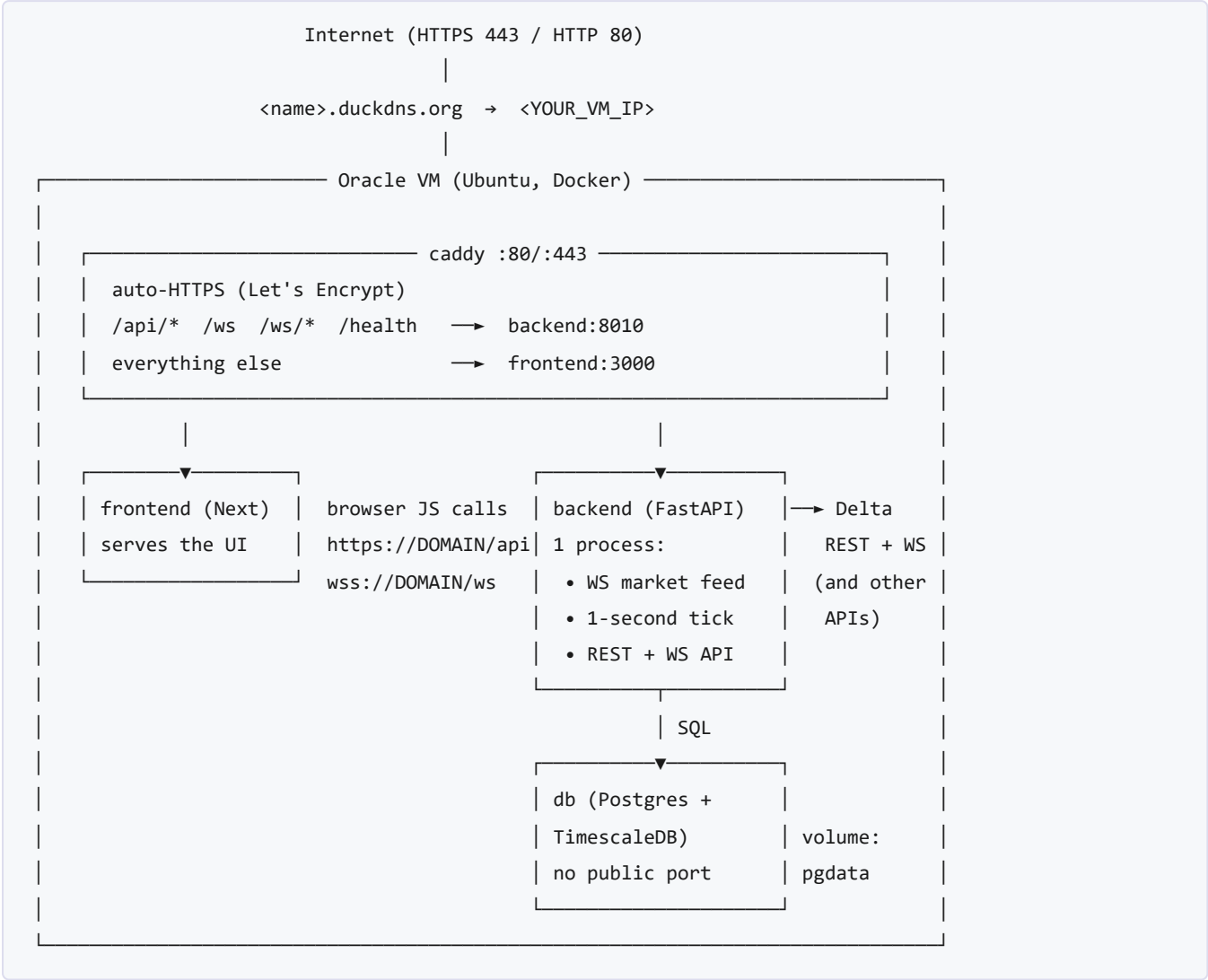
1. TL;DR — the stack

Layer	What I use	Why
Backend	Python 3.12, **FastAPI** + uvicorn, **uv** (package manager), pydantic v2	Async, fast, great for WebSockets
DB access	SQLAlchemy 2 (async, asyncpg) + **Alembic** migrations	Schema versioned in git, auto-applied on start
Market data	`httpx` (Delta REST) + `websockets` (Delta WS feed)	One persistent WS, in-memory ticker cache
Maths	numpy / scipy (Black-Scholes, Greeks)	
Frontend	**Next.js 16** (App Router), React 19, TypeScript, Tailwind v4	
UI state / charts	zustand, **lightweight-charts** (TradingView's free lib)	
Database	**PostgreSQL 16 + TimescaleDB 2.17** (Docker image `timescale/timescaledb:2.17.2-pg16`)	Time-series tables (hypertables) for P&L history
Auth	Supabase (Google login + 2FA) — *identity only*, data stays in my Postgres	Free, no auth code to write
Reverse proxy / HTTPS	**Caddy 2**	Free Let's Encrypt certs automatically, WebSockets just work
Packaging	**Docker Compose** (4 containers)	One command to deploy
Hosting	**Oracle Cloud Always Free** VM (VM.Standard.E2.1.Micro, 1 GB RAM, Ubuntu 22.04) + 2 GB swap	Free forever
Domain	**DuckDNS** free subdomain	Free, HTTPS needs a hostname
Monitoring	healthchecks.io (free) pinged by a cron job	Email alert if the site dies
CI	GitHub Actions (tests + build on every PR)	

You don't have to use the same languages. The *deployment pattern* (VM + Docker Compose + Caddy + Postgres + DuckDNS) works for any backend (Node/Express, Django, Go…) and any frontend (Next.js, React/Vite, or a single static HTML page).

---

2. Architecture



Key ideas

- **One domain, path-based routing.** The browser only ever talks to `https://<domain>`. Caddy sends `/api`, `/ws`, `/health` to the backend and everything else to the frontend. No CORS headaches, one certificate.
- **Only Caddy is exposed.** DB, backend and frontend have no public ports; they talk over the Docker network by service name (`db:5432`, `backend:8010`).
- **Backend data flow:** Delta WebSocket → in-memory ticker cache → a loop every 1 s recomputes P&L / runs stop-loss exits → writes to Postgres → pushed to the browser over WebSocket.
- **Background loops** start with the app (FastAPI `lifespan`): the market-data WS ingestor (auto-reconnect, 20 s ping), the 1 s ticker, and a live-account sync.
- **Single backend worker on purpose.** The tick loop and caches live in process memory — two workers would double-tick and split state. Scaling would mean moving the loop to its own process and caches to Redis. Not needed at this size.
- **Where state lives:** Postgres = source of truth (positions, balances, logs, history). Process memory = live prices + last 15 min of 1 s data (bounded). Browser = only UI prefs.

---

3. Repo layout to mirror

```
<repo>/
├── backend/
│   ├── Dockerfile
│   ├── docker-entrypoint.sh      # run migrations, then start server
│   ├── .env.prod                 # secrets — on the VM only, NOT in git
│   └── app/ ...                  # must expose GET /health
├── frontend/                     # Next.js / React app  (or web/ for a static HTML page)
│   └── Dockerfile
├── docker-compose.yml            # local dev (optional)
├── docker-compose.prod.yml       # production stack
├── Caddyfile
├── .env.prod.example             # names only, committed
├── .env                          # real values — on the VM only, NOT in git
└── .gitignore                    # must include .env, *.env.prod, .env.*  (keep *.example)
```

---

4. Template files

4.1 `docker-compose.prod.yml`

```
services:
  db:
    image: timescale/timescaledb:2.17.2-pg16   # or postgres:16-alpine if you don't need Timescale
    # Pin Postgres memory for a SHARED 1 GB VM (see Gotchas #1). Without this, the
    # auto-tuner grabs ~300 MB and the whole box starts swapping.
    command: [postgres,
      -c, shared_buffers=96MB, -c, maintenance_work_mem=32MB,
      -c, effective_cache_size=256MB, -c, work_mem=4MB, -c, max_connections=30]
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-app}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
      POSTGRES_DB: ${POSTGRES_DB:-app}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-app} -d ${POSTGRES_DB:-app}"]
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped

  backend:
    build: ./backend
    env_file: ./backend/.env.prod
    environment:
      DATABASE_URL: postgresql+asyncpg://${POSTGRES_USER:-
app}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB:-app}
      APP_ENV: production
      CORS_ORIGINS: https://${DOMAIN:?set DOMAIN in .env}
    depends_on:
      db:
        condition: service_healthy
    mem_limit: 512m          # a leak restarts only the backend instead of freezing the VM
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      args:                  # baked in at build time (see Gotchas #4)
        NEXT_PUBLIC_API_BASE: https://${DOMAIN}
        NEXT_PUBLIC_WS_BASE: wss://${DOMAIN}
    restart: unless-stopped
```

```
  caddy:
    image: caddy:2-alpine
    ports: ["80:80", "443:443"]
    environment:
      DOMAIN: ${DOMAIN}
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddydata:/data      # keeps the TLS cert across restarts — don't delete
      - caddyconfig:/config
    depends_on: [frontend, backend]
    restart: unless-stopped

volumes:
  pgdata:
  caddydata:
  caddyconfig:
```

4.2 `Caddyfile`

```
{$DOMAIN} {
    @backend path /api/* /ws /ws/* /health
    handle @backend {
        reverse_proxy backend:8010
    }
    handle {
        reverse_proxy frontend:3000
    }
}
```

Static single-page variant (your dashboard is one HTML file + JS): delete the `frontend` service, mount the folder into Caddy (`- ./web:/srv:ro`) and replace the last `handle` with:

```
    handle {
        root * /srv
        file_server
    }
```

In the page's JS use relative URLs — `fetch('/api/...')` and `new WebSocket(` wss://${location.host}/ws `)` — so nothing is hard-coded to a domain.

4.3 Backend `Dockerfile` (Python / FastAPI + uv)

```
# Single process only — background loops + in-memory caches are per-process.
FROM python:3.12-slim
ENV PYTHONUNBUFFERED=1
RUN pip install --no-cache-dir uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY . .
RUN chmod +x docker-entrypoint.sh
EXPOSE 8010
CMD ["./docker-entrypoint.sh"]
```

`backend/docker-entrypoint.sh`:

```
#!/bin/sh
set -e
uv run alembic upgrade head          # apply DB migrations on every start
exec uv run uvicorn app.main:app --host 0.0.0.0 --port 8010
```

Health endpoint (FastAPI):

```
@app.get("/health")
async def health():
    return {"status": "ok", "version": "0.1.0", "env": settings.app_env}
```

(Node backend? Same idea: `node:20-alpine`, `npm ci --omit=dev`, run migrations (Prisma/Knex) then `node server.js` on port 8010.)

4.4 Frontend `Dockerfile` (Next.js)

```
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ARG NEXT_PUBLIC_API_BASE
ARG NEXT_PUBLIC_WS_BASE
ENV NEXT_PUBLIC_API_BASE=$NEXT_PUBLIC_API_BASE \
    NEXT_PUBLIC_WS_BASE=$NEXT_PUBLIC_WS_BASE \
    NODE_ENV=production
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

---

5. Environment variables (names only)

Root `.env` (read by docker compose — on the VM only):

```
DOMAIN=<name>.duckdns.org
POSTGRES_USER=app
POSTGRES_PASSWORD=<long random string>
POSTGRES_DB=app
# + any public build-time values, e.g. SUPABASE_URL / SUPABASE_ANON_KEY
```

`backend/.env.prod` (secrets — on the VM only). What I use:

- Delta: `DELTA_API_KEY`, `DELTA_API_SECRET` (+ optional `DELTA_API_BASE`, `DELTA_WS_BASE`; India = `https://api.india.delta.exchange`, `wss://socket.india.delta.exchange`)
- Auth: `SUPABASE_URL`, `SUPABASE_JWT_AUD`
- App: `SECRET_VAULT_KEY` (encrypts stored user API keys), `ADMIN_EMAIL`, `LIVE_TRADING_ENABLED=false`, `RATE_LIMIT_ENABLED`, `RATE_LIMIT_PER_MIN`
- Alerts (optional): Telegram bot token / chat id

Set by compose, not by you: `DATABASE_URL`, `APP_ENV`, `CORS_ORIGINS`.

Rules: commit only `*.example` files with empty values; generate secrets with `openssl rand -hex 32`; use **read-only / trade-disabled** exchange API keys unless you really need trading; whitelist the VM IP on the Delta API key if you can.

---

6. Step-by-step deployment

Step 1 — Create the VM (Oracle Cloud Always Free)

1. Sign up at cloud.oracle.com (card needed for verification, not charged on Always Free).
2. **Compute → Instances → Create instance.**
   - Image: **Ubuntu 22.04**.
   - Shape: `VM.Standard.E2.1.Micro` (1 GB, always available) — or **Ampere A1** (ARM, up to 4 OCPU / 24 GB free) if it has capacity in your region; A1 is much roomier but often "out of capacity".
   - Add your SSH public key (`ssh-keygen -t ed25519 -f ~/.ssh/oracle_key`).
3. Note the **public IP**. Test: `ssh -i ~/.ssh/oracle_key ubuntu@<YOUR_VM_IP>` .

Step 2 — Open ports 80 and 443 (Oracle has TWO firewalls)

1. **Cloud side:** Instance → Subnet → Security List → *Add Ingress Rules*: source `0.0.0.0/0` , TCP, destination ports `80` and `443` .
2. **On the VM** (Oracle's Ubuntu image ships strict iptables):

   ```
   sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80  -j ACCEPT
   sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
   sudo netfilter-persistent save
   ```

   Do **not** open 5432/8010/3000 — only Caddy is public.

Step 3 — Add 2 GB swap (mandatory on a 1 GB box)

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swap.conf && sudo sysctl --system
```

Step 4 — Install Docker

```
sudo apt update && sudo apt install -y docker.io docker-compose-plugin git
sudo usermod -aG docker $USER      # then log out and back in
docker compose version             # sanity check
```

Step 5 — Free domain with DuckDNS

1. duckdns.org → sign in → add a subdomain, e.g. `myname-trader` → `myname-trader.duckdns.org` .
2. Set its IP to `<YOUR_VM_IP>` → *update ip*.
3. Check: `ping myname-trader.duckdns.org` shows your VM IP. (Oracle public IPs are static while the instance exists, so no updater cron is needed.)

Step 6 — Get the code onto the VM

Pick one:

- **Public repo, or private repo with a deploy key** (easiest to redeploy):

  ```
  git clone git@github.com:<you>/<repo>.git ~/app
  ```

  (Private: create a key on the VM, add the `.pub` as a *read-only Deploy key* in GitHub repo settings.)
- **What I do (private repo, nothing on the VM touches GitHub)** — from my laptop:

  ```
  git archive --format=tar.gz -o /tmp/app.tgz main
  scp -i ~/.ssh/oracle_key /tmp/app.tgz ubuntu@<YOUR_VM_IP>:/tmp/
  ssh -i ~/.ssh/oracle_key ubuntu@<YOUR_VM_IP> 'mkdir -p ~/app && cd ~/app && tar xzf
  /tmp/app.tgz'
  ```

  Note: no `.git` on the server — git archive stamps every file with the commit time, which is how you can tell which commit is deployed (`ls -l --time-style=full-iso`).

Step 7 — Create the secret files on the VM

```
cd ~/app
cp .env.prod.example .env               && nano .env              # DOMAIN, POSTGRES_*
cp backend/.env.prod.example backend/.env.prod && nano backend/.env.prod
chmod 600 .env backend/.env.prod
```

These are never in git and survive redeploys (the tarball doesn't contain them).

Step 8 — Launch

```
cd ~/app
docker compose -f docker-compose.prod.yml up -d --build > ~/deploy.log 2>&1; tail ~/deploy.log
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f backend     # Ctrl-C to stop following
```

The first build on a 1 GB VM can take 10–20 min (the Next.js build is the slow part). Caddy gets the TLS certificate on the first request (~30 s).

Step 9 — Verify

```
curl https://<name>.duckdns.org/health     # → {"status":"ok",...}
```

Open `https://<name>.duckdns.org` in the browser; check the WebSocket connects (DevTools → Network → WS).

Step 10 (only if using Supabase login)

Supabase → Authentication → URL Configuration: Site URL `https://<name>.duckdns.org`, Redirect URL `https://<name>.duckdns.org/**`.

---

7. Day-2 operations

Task	Command
Redeploy after changes	repeat Step 6 (archive+scp, or `git pull`), then the Step 8 `up -d --build`
Logs	`docker compose -f docker-compose.prod.yml logs --tail 200 backend`
Restart one service	`docker compose -f docker-compose.prod.yml restart backend`
Memory check	`free -m`, `docker stats --no-stream`
DB shell	`docker compose -f docker-compose.prod.yml exec db psql -U app app`
Backup	`docker compose -f docker-compose.prod.yml exec db pg_dump -U app app > backup-$(date +%F).sql`
Disk cleanup	`docker image prune -f` (old images pile up after rebuilds)

Uptime alert (free): create a check on healthchecks.io (period 2 min, grace 5 min), put its ping URL in `/etc/hc-ping.url` (`chmod 600`), and cron this every 2 min (`*/2 * * * * /usr/local/bin/hc-ping.sh`):

```
#!/usr/bin/env bash
URL="$(cat /etc/hc-ping.url 2>/dev/null | tr -d '[:space:]')"
[ -z "$URL" ] && exit 0
if curl -fsS -m 10 https://<name>.duckdns.org/health | grep -q '"status":"ok"'; then
  curl -fsS -m 10 --retry 3 "$URL" >/dev/null
fi
```

If the site dies, pings stop and you get an email.

CI (GitHub Actions): on every PR run backend tests (with a Postgres service container), frontend type-check + tests + `next build`. Deploy stays manual — that's fine at this size.

---

8. Gotchas I learned the hard way

1. **1 GB RAM: swap thrash, not a crash.** The site went "stale / can't reach server" with only ~34% CPU but high load average. Cause: TimescaleDB auto-tuned `shared_buffers` to ~305 MB as if it owned the box → everything swapped → the async backend stalled for seconds → `/health` timed out and the exchange WebSocket ping was missed. Fix: the `-c` memory flags on `db` + `mem_limit` on backend (both in the template). Restarting only helps for minutes.
2. **Bound every in-memory buffer.** My 1-second history was an unbounded 24 h list; now it's `deque(maxlen=900)` (15 min). Backend sits at ~200–340 MB.
3. **Building on the VM is heavy.** Next.js build needs the swap and takes 10–20 min. Batch changes into one deploy. Alternatives: host the frontend free on Vercel/Cloudflare Pages, or build images in GitHub Actions and `docker pull` them on the VM.
4. **`NEXT_PUBLIC_*` values are baked in at build time.** Changing the domain = rebuild the frontend. (Static HTML with relative URLs avoids this entirely.)
5. **Wait for the build to finish before testing.** "Up 3 hours" in `docker ps` right after a deploy means you're still looking at the *old* container; check `~/deploy.log`.
6. **Time-series tables grow forever.** Add a TimescaleDB retention/compression policy (or a cleanup job) from day one, or the 1 GB box fills up.
7. **One backend worker** when you have background loops/in-memory state (see §2).
8. **Never lose your encryption key** (`SECRET_VAULT_KEY`) — stored encrypted secrets become unreadable. Back it up somewhere safe outside the VM.
9. **Don't put exchange keys with trading/withdraw permission on a free VM** unless you mean to. Keep a kill switch env var (`LIVE_TRADING_ENABLED=false`).
10. **Keep the `caddydata` volume.** Deleting it re-requests certificates; Let's Encrypt rate-limits repeated requests.

---

9. Checklist

- [ ] App runs locally with `docker compose up` and `GET /health` returns ok
- [ ] `.env` files gitignored; `.env*.example` committed with names only
- [ ] Oracle VM created, SSH works
- [ ] Ports 80/443 open in Security List **and** iptables
- [ ] 2 GB swap on
- [ ] Docker + compose plugin installed
- [ ] DuckDNS subdomain → VM IP
- [ ] Code on VM, `.env` + `backend/.env.prod` filled, `chmod 600`
- [ ] `docker compose -f docker-compose.prod.yml up -d --build` done, all 4 containers Up
- [ ] `https://<name>.duckdns.org/health` ok, padlock shows, WebSocket connects
- [ ] healthchecks.io ping cron installed
- [ ] DB backup command tested once